Forma–1 API

Bemutató REST API · Express 5 · egyetlen fájl · saját middleware-ek egyenként és láncban

Ebben a feladatban egy kicsi, három végpontos API-t készítesz Forma–1-es világbajnokokról. Maga az API egyszerű — a lényeg a **middleware**: saját függvényeket írsz, amelyek a kérés útjába állnak, és eldöntik, mehet-e tovább.

1. Mit készítünk?

Végpont	Mit tesz?	Saját middleware-ek
`GET /api/versenyzok`	az összes versenyző	–
`GET /api/versenyzok/:id`	egy versenyző	`ervenyesId → versenyzoKeresese`
`POST /api/versenyzok`	új versenyző	`apiKulcs → ervenyesVersenyzo`

Mindhárom előtt lefut még egy ötödik middleware, a `naplozo`: ez minden kérést kiír a terminálra.

Egy versenyző öt mezőből áll:

Mező	Típus	Jelentés
`id`	`szám`	egyedi azonosító — a szerver adja
`nev`	`szöveg`	a versenyző neve
`nemzetiseg`	`szöveg`	a nemzetisége
`cimek`	`szám`	hány világbajnoki címet nyert
`elsoCim`	`szám`	melyik évben nyerte az elsőt

2. Mi az a middleware?

Olyan függvény, amelyet az Express a kérés és a válasz közé fűz. Három paramétert kap:

```
function middlewareNeve(req, res, next) {
  // megvizsgál vagy előkészít valamit
  next();
}
```

- `req` — a beérkező kérés. A middleware olvashatja, és új mezőket is tehet rá.
- `res` — a válasz. Ha a middleware hibát talál, ezen keresztül maga válaszol.
- `next` — függvény. Ha a middleware meghívja, a kérés továbbmegy a lánc következő elemére.

Egy middleware tehát kétféleképpen végezhet:

Mit tesz?	Mi történik a kéréssel?
meghívja a `next()`-et	továbbmegy a következő middleware-re, végül a végpont kezelőjére
válaszol, és nem hívja a `next()`-et	a kérés itt véget ér; a lánc többi eleme le sem fut

Az egy versenyzőt lekérő végpont útja így néz ki:

```
kérés -> naplozo -> express.json() -> ervenyesId -> versenyzoKeresese -> kezelő
                                          |                |
                                          v                v
                                   400, ha rossz     404, ha nincs
                                   az azonosító      ilyen versenyző
```

Az `express.json()` is middleware — csak nem mi írtuk, hanem az Express adja.

3. A projekt létrehozása

1. Hozz létre egy **forma1-api** mappát, nyisd meg a VS Code-ban, és nyiss benne egy terminált.
2. `npm init -y` — létrehozza a `package.json` fájlt az alapértelmezésekkel.
3. A `package.json`-ben írd át a `"type"` értékét `"module"`-ra — e nélkül az `import` kulcsszó hibát adna. A `"description"` legyen `"Bemutató REST API: saját middleware-ek"`.
4. A `scripts` szakasz `test` sorát cseréld le erre a kettőre:

```
"start": "node index.js",
"dev": "node --watch index.js"
```

5. `npm install express` — telepíti az Expresst a `node_modules` mappába.
6. Ha Gitet használsz, a `.gitignore` fájlba írd be: `node_modules/`.

A `dev` parancs a `--watch` miatt minden mentéskor újraindítja a szervert. Fejlesztés közben ezt használjuk: `npm run dev`.

4. Az alap

4.1 Az index.js eleje

Hozd létre az **index.js** fájlt:

```
import express from "express";

const app = express();
const PORT = 3000;
```

- `import express from "express";` — importáld az Expresst.
- `const app = express();` — elkészíti az alkalmazást; ehhez fűzzük a middleware-eket és a végpontokat.
- `const PORT = 3000;` — a port, amelyen a szerver a kéréseket várja.

4.2 Az adatok

Utána jöjjenek az adatok:

```
const adatok = {
  kovetkezoId: 16,
  versenyzok: [
    { id: 1, nev: "Juan Manuel Fangio", nemzetiseg: "argentin", cimek: 5, elsoCim: 1951 },
    { id: 2, nev: "Alberto Ascari", nemzetiseg: "olasz", cimek: 2, elsoCim: 1952 },
    { id: 3, nev: "Jack Brabham", nemzetiseg: "ausztrál", cimek: 3, elsoCim: 1959 },
    { id: 4, nev: "Jim Clark", nemzetiseg: "brit", cimek: 2, elsoCim: 1963 },
    { id: 5, nev: "Jackie Stewart", nemzetiseg: "brit", cimek: 3, elsoCim: 1969 },
    { id: 6, nev: "Emerson Fittipaldi", nemzetiseg: "brazil", cimek: 2, elsoCim: 1972 },
    { id: 7, nev: "Niki Lauda", nemzetiseg: "osztrák", cimek: 3, elsoCim: 1975 },
    { id: 8, nev: "Nelson Piquet", nemzetiseg: "brazil", cimek: 3, elsoCim: 1981 },
    { id: 9, nev: "Alain Prost", nemzetiseg: "francia", cimek: 4, elsoCim: 1985 },
    { id: 10, nev: "Ayrton Senna", nemzetiseg: "brazil", cimek: 3, elsoCim: 1988 },
    { id: 11, nev: "Nigel Mansell", nemzetiseg: "brit", cimek: 1, elsoCim: 1992 },
    { id: 12, nev: "Michael Schumacher", nemzetiseg: "német", cimek: 7, elsoCim: 1994 },
    { id: 13, nev: "Mika Häkkinen", nemzetiseg: "finn", cimek: 2, elsoCim: 1998 },
    { id: 14, nev: "Kimi Räikkönen", nemzetiseg: "finn", cimek: 1, elsoCim: 2007 },
    { id: 15, nev: "Sebastian Vettel", nemzetiseg: "német", cimek: 4, elsoCim: 2010 },
  ],
};
```

- `kovetkezoId` — a következő új versenyző azonosítója; minden felvételnél eggyel nő.
- `versenyzok` — tizenöt világbajnok, az első címük sorrendjében.

Az adatok a memóriában vannak: minden újraindításkor — és a `--watch` miatt minden mentéskor — visszaáll a fenti tizenöt versenyző.

4.3 Az indítás

A fájl **legvégére**:

```
app.listen(PORT, (hiba) => {
  if (hiba) throw hiba;
  console.log(`A szerver fut: http://localhost:${PORT}`);
});
```

- `app.listen(…)` — elindítja a szervert. A callback akkor fut le, amikor a szerver elindult — vagy amikor nem sikerült.
- `if (hiba) throw hiba;` — foglalt portnál leállítja a programot, és kiírja az okát.

Ez végig az utolsó rész marad: minden, amit ezután írsz, elé kerül.

5. Az első middleware: a naplózó

5.1 A függvény

Az adatok alá:

```
function naplozo(req, res, next) {
  const ido = new Date().toLocaleTimeString("hu-HU");
  console.log(`${ido} ${req.method} ${req.originalUrl}`);
  next();
}
```

- `const ido = …` — a pillanatnyi idő magyar formában, például `7:36:29`.

- `req.method` — a kérés metódusa: `GET`, `POST`…
- `req.originalUrl` — a kért cím, a `?` utáni résszel együtt.
- `next();` — a naplózó nem dönt semmiről, csak megjegyzi a kérést, ezért mindig továbbengedi.

5.2 Bekapcsolás minden kérésre

A függvény alá:

```
app.use(naplozo);
app.use(express.json());
```

- `app.use(fv)` — a middleware-t **minden** kérés elé fűzi, akármelyik végpontnak szól.
- `express.json()` — a JSON-törzset objektummá alakítja, és a `req.body`-ba teszi.
- A sorrend számít. A naplózó áll elöl, így az elromlott JSON-t küldő kérés is bekerül a naplóba: az `express.json()` hibát ad rá, de akkor a naplózó már lefutott.

5.3 Az első végpont

Az `app.use` sorok alá:

```
app.get("/api/versenyzok", (req, res) => {
  res.json(adatok.versenyzok);
});
```

- Ez a végpont nem kap saját middleware-t; csak a két `app.use`-os fut le előtte.
- `res.json(…)` — JSON-ként küldi el a listát, 200-as kóddal.

Indítsd el a szervert (`npm run dev`), és nyisd meg a böngészőben a **http://localhost:3000/api/versenyzok** címet: megjelenik a tizenöt versenyző. A terminálon:

```
A szerver fut: http://localhost:3000
7:36:29 GET /api/versenyzok
```

Minden újabb kérés újabb sort ír ki — a naplózó dolgozik.

6. Két middleware láncban: egy versenyző

6.1 Az azonosító ellenőrzése

A `naplozo` függvény alá:

```
function ervenyesId(req, res, next) {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id < 1) {
    return res.status(400).json({ hiba: "Az azonosító pozitív egész szám legyen." });
  }
  req.versenyzoId = id;
  next();
}
```

- `Number(req.params.id)` — az útvonal `:id` része mindig szöveg; számmá alakítjuk.
- `!Number.isInteger(id) || id < 1` — ha nem pozitív egész szám (`abc`, `0`, `2.5`)…
- `return res.status(400).json(…)` — …a middleware maga válaszol **400**-zal. A `return` miatt a függvény itt véget ér, a `next()` nem fut le, és a kérés nem megy tovább.
- `req.versenyzoId = id;` — a számmá alakított értéket **rátesszük a kérésre**. A lánc következő elemei ugyanezt a `req` objektumot kapják, így ott már készen találják.

6.2 A versenyző megkeresése

Utána:

```
function versenyzoKeresese(req, res, next) {
  const versenyzo = adatok.versenyzok.find((v) => v.id === req.versenyzoId);
  if (!versenyzo) {
    return res.status(404).json({ hiba: "Nincs ilyen azonosítójú versenyző." });
  }
  req.versenyzo = versenyzo;
  next();
}
```

- `req.versenyzoId` — ezt az előző middleware tette a kérésre. Ez a függvény tehát **számít rá**, hogy az `ervenyesId` előtte lefutott.
- `find(…)` — az első versenyző, akinek egyezik az azonosítója, vagy `undefined`.
- Ha nincs ilyen: **404**, és a kérés itt véget ér.
- `req.versenyzo = versenyzo;` — a talált versenyzőt is rátesszük a kérésre, a kezelőnek.

6.3 A végpont

Az első végpont alá:

```
app.get("/api/versenyzok/:id", ervenyesId, versenyzoKeresese, (req, res) => {
  res.json(req.versenyzo);
});
```

- A vesszővel felsorolt függvények **balról jobbra** futnak le, a kezelő az utolsó.
- Ezek csak ennél a végpontnál futnak le — az első végpontot nem érintik.
- A kezelő egyetlen sor: mire ideér a kérés, az azonosítót már megvizsgálta az `ervenyesId`, és a versenyzőt is megtalálta a `versenyzoKeresese`.

A válaszok:

Kérés	Válasz	Mi futott le?
`/api/versenyzok/12`	200	mindkét middleware és a kezelő — Michael Schumacher adatai
`/api/versenyzok/abc`	400	csak az `ervenyesId`; a `versenyzoKeresese` el sem indult
`/api/versenyzok/99`	404	az `ervenyesId`, majd a `versenyzoKeresese`; a kezelő nem

A lánc sorrendje nem mindegy. Ha a kettőt megcseréled, a `versenyzoKeresese` fut le először, amikor a `req.versenyzoId` még nincs beállítva. Ilyenkor a létező versenyzőre is **404** jön, a hibás azonosítóra pedig 404 a 400 helyett.

7. Postman

Az új versenyző felvételéhez POST kérés kell, ezt a böngészőből nem lehet elküldeni.

1. Hozz létre egy gyűjteményt **Forma–1 API** néven.
2. Vegyél fel egy környezetet egy `host` változóval: `http://localhost:3000/api`, és válaszd ki.
3. A törzs a **Body** → **raw** → **JSON** fülre kerül.
4. A fejléceket a **Headers** fülön adhatod meg.

8. Védett végpont: új versenyző

8.1 A kulcs ellenőrzése

A `versenyzoKeresese` függvény alá:

```
function apiKulcs(req, res, next) {
  if (req.get("X-Api-Kulcs") !== "pitstop") {
    return res.status(401).json({ hiba: "Hiányzó vagy hibás API-kulcs." });
  }
  next();
}
```

- `req.get("X-Api-Kulcs")` — kiolvassa a kérés egyik fejlécét. Ha a kliens nem küldte, az értéke `undefined`.
- Ha a kulcs hiányzik vagy nem egyezik: **401**, vagyis „azonosítsd magad”.
- Ha egyezik, a kérés továbbmehet.

Ez egy egyszerűsített védelem a bemutatóhoz: valódi API-ban a kulcs nem a kódban áll, és mindenkinek saját kulcsa van.

8.2 A végpont — egyelőre egy middleware-rel

A második végpont alá:

```
app.post("/api/versenyzok", apiKulcs, (req, res) => {
  const { nev, nemzetiseg, cimek, elsoCim } = req.body;
  const uj = { id: adatok.kovetkezoId, nev, nemzetiseg, cimek, elsoCim };

  adatok.kovetkezoId += 1;
  adatok.versenyzok.push(uj);

  res.status(201).json(uj);
});
```

- `apiKulcs` — csak ez az egy middleware áll a kezelő előtt.
- `const { … } = req.body;` — a négy mezőt kiolvassuk a törzsből.
- `id: adatok.kovetkezoId` — az azonosítót a szerver adja; utána a számláló eggyel nő.
- `res.status(201)` — **201**: létrejött.

Próbáld ki Postmanben — **POST** `{{host}}/versenyzok`, a törzs:

```
{ "nev": "Nico Rosberg", "nemzetiseg": "német", "cimek": 1, "elsoCim": 2016 }
```

- **X-Api-Kulcs** fejléc nélkül: **401**.
- A Headers fülön `X-Api-Kulcs` = `pitstop` fejléccel: **201**, Nico Rosberg a 16-os azonosítót kapja.

Most küldd el a kulccsal, de üres törzzsel: `{}`. A válasz **201** — és létrejött egy versenyző, akinek csak azonosítója van: `{"id":17}`. A kulcs megvan, az adatokat viszont senki nem nézte meg.

8.3 Az adatok ellenőrzése

Az `apiKulcs` függvény alá:

```
function ervenyesVersenyzo(req, res, next) {
  const { nev, nemzetiseg, cimek, elsoCim } = req.body;
  if (!nev || !nemzetiseg || !Number.isInteger(cimek) || !Number.isInteger(elsoCim)) {
    return res.status(400).json({ hiba: "A nev, a nemzetiseg, a cimek és az elsoCim kötelező." });
  }
  next();
}
```

- A név és a nemzetiség nem lehet üres, a két szám egész szám legyen.
- Ha bármelyik nem stimmel: **400**, és a kezelő nem fut le.

8.4 A lánc

Fűzd be a második middleware-t a végpont elé — csak az első sor változik:

```
app.post("/api/versenyzok", apiKulcs, ervenyesVersenyzo, (req, res) => {
```

A válaszok:

Kérés	Válasz	Ki állította meg?
kulcs nélkül, jó törzzsel	401	`apiKulcs`
kulccsal, `{}` törzzsel	400	`ervenyesVersenyzo`
kulccsal, jó törzzsel	201	senki — a kezelő létrehozta
kulcs nélkül, `{}` törzzsel	401	`apiKulcs` — az adatokig el sem jut

Az `apiKulcs` áll elöl, és ennek oka van. Aki nem azonosította magát, annak a kérésével nem foglalkozunk tovább — azt sem tudja meg, milyen adatokat várna a végpont. Fordított sorrendben az idegen is megkapná a **400**-as hibaüzenetet a kötelező mezők listájával.

9. A middleware-ek három helye

Hogyan fűzöd fel?	Mikor fut le?	A példában
`app.use(mw)`	minden kérésnél	`naplozo`, `express.json()`
`app.post(út, mw, kezelő)`	csak ennél a végpontnál	`apiKulcs` a 8.2-ben
`app.get(út, mw1, mw2, kezelő)`	ennél a végpontnál, balról jobbra	`ervenyesId` → `versenyzoKeresese`, `apiKulcs` → `ervenyesVersenyzo`

10. A kész fájl sorrendje

1. az Express importálása, az `app` és a `PORT`
2. az `adatok` objektum
3. az öt middleware-függvény: `naplozo`, `ervenyesId`, `versenyzoKeresese`, `apiKulcs`, `ervenyesVersenyzo`
4. `app.use(naplozo)`, majd `app.use(express.json())`

5. a három végpont
6. `app.listen(…)` — mindig a legvégén

11. Sorvezető

Amit használsz	Mit csinál?
`function mw(req, res, next)`	egy middleware; a `next()` hívásával továbbengedi a kérést
`app.use(mw)`	a middleware minden kérésnél lefut
`app.get(út, mw1, mw2, kezelő)`	a middleware-ek csak ennél a végpontnál futnak, balról jobbra
`return res.status(kód).json(…)`	a middleware maga válaszol, és a lánc megáll
`req.valami = érték`	adat továbbadása a lánc következő elemeinek
`req.get("Fejléc-Neve")`	egy kérésfejléc értéke, vagy `undefined`
`req.method, req.originalUrl`	a kérés metódusa és címe
`new Date().toLocaleTimeString("hu-HU")`	a pillanatnyi idő magyar formában

12. Ha nem működik

Tünet	Mi lehet a baj?
a kérés válasz nélkül áll	egy middleware-ből kimaradt a `next()`
a terminálon nem jelennek meg a kérések	az `app.use(naplozo)` a végpontok **után** áll
a létező versenyzőre is 404	a láncban a `versenyzoKeresese` az `ervenyesId` előtt áll
mindig 401, pedig küldöd a kulcsot	a fejléc neve vagy értéke el van gépelve, vagy nem a **Headers** fülön adtad meg
401, mégis létrejött a versenyző	az `apiKulcs`-ból kimaradt a `return`
500, `Cannot destructure property 'nev' of 'req.body'`	a Postmanben nem **raw** → **JSON** a törzs

13. Próbáld ki

Mindegyiket egy működő változaton érdemes kipróbálni, utána állítsd vissza.

1. Vedd ki a `naplozo`-ból a `next();` sort, és kérd le a listát. A terminálon megjelenik a kérés, de a válasz **soha nem érkezik meg**: a lánc a naplózóban elakadt.
2. Tedd az `app.use(naplozo);` sort a végpontok **után**. A három végpontra érkező kérések eltűnnek a naplóból; csak a nem létező címekre (`/api/nincsilyen`) jelenik meg sor — ezeknél a kérés végigmegy a láncon, és eléri a naplózót.
3. Tedd a naplózót az `express.json()` **mögé**, és küldj elromlott JSON-t. A válasz ugyanaz a hibaoldal, de a naplóból hiányzik a kérés sora.

4. Cseréld meg a `GET /api/versenyzok/:id` láncában a két middleware-t. A `/api/versenyzok/12` **404**-et ad, az `/abc` is 404-et — a 400 helyett.
5. Vedd ki a `return` szót az `apiKulcs`-ból, és küldj kulcs nélkül egy jó törzset. A válasz **401** — a versenyző mégis bekerül a listába, a terminálon pedig megjelenik: `Cannot set headers after they are sent to the client`. A middleware válaszolt, de utána tovább is engedte a kérést.
6. Cseréld meg a POST láncában a két middleware-t, és küldj kulcs nélkül `{}` törzset. Kulcs nélkül is **400**-at kapsz, a kötelező mezők listájával.

14. Önálló feladatok

Ezekhez nincs kész kód — az eddigiek alapján meg tudod írni őket.

1. **Késésszámláló.** Írj egy `szamlalo` nevű middleware-t, amely minden kérésnél eggyel növel egy számlálót, és kiírja: `3. kérés`. Minden kérésnél fusson le.
2. **Adattisztító.** Írj egy `adatTisztito` nevű middleware-t, amely levágja a `nev` és a `nemzetiseg` elejéről és végéről a szóközöket (`trim()`). Tedd a POST láncába harmadiknak. Hová kerüljön, hogy a csak szóközökből álló név **ne** menjen át az ellenőrzésen?
3. **Védett lekérdezés.** Az egy versenyzőt lekérő végpontot is védd le az `apiKulcs`-csal. Milyen sorrendben álljon a három middleware, hogy kulcs nélkül az `/api/versenyzok/abc` is **401**-et adjon?

15. Ellenőrzőlista

1. A terminálon minden kérésről megjelenik egy sor, az elromlott JSON-t küldőről is.
2. A `/api/versenyzok` tizenöt versenyzőt ad.
3. A `/api/versenyzok/12` **200**-at ad, az `/abc` **400**-at, a `/99` **404**-et.
4. A POST kulcs nélkül **401**-et ad; kulccsal és hiányos törzzsel **400**-at; kulccsal és jó törzzsel **201**-et.
5. Kulcs nélkül, hiányos törzzsel is **401** a válasz.
6. Minden middleware-ben a válasz előtt `return` áll, a végén `next()`.